

Pedro Pathing Tuning Guide (Mecanum + Pinpoint)

Oct 9, 2026 · @Inkeun Kim

Overview

Tuning Pedro Pathing takes five steps, done in order, and AutoTune does most of the work. AutoTune is a web page hosted by the Robot Controller: you run each procedure from a browser, then paste the Java it generates into `Constants.java`.

This guide assumes a mecanum drivetrain and a goBILDA Pinpoint localizer. Other localizers (OctoQuad, OTOS, two- or three-wheel) follow the same flow with a different tuner. It follows the Pedro 3 documentation.

1. **Drivetrain:** tell Pedro your motor names and spin directions.
2. **Localizer:** tune the Pinpoint so the robot knows where it is on the field.
3. **Foresight:** tune the path-following algorithm (velocities, braking, kP values).
4. **Constants:** combine all three configs and build the `Follower`.
5. **Test:** verify the finished Follower with the Tests procedure.

Before you start

Finish the Pedro Pathing installation first, then confirm you can build and deploy to the Robot Controller. Every procedure below follows the same loop.

1. Add a `@Tuner` method to your `Tuning` class.
2. Build and upload the code to the Robot Controller.
3. While connected to the Robot Controller, open `http://192.168.43.1:10158` in a browser.
4. Pick the procedure and follow the on-screen prompts.
5. When it finishes, open the **Java** tab and copy the generated config into `Constants.java`.

Each tuner is a static method that returns a `Procedure`, for example:

```
@Tuner
public static Procedure mecanumTuner() {
    return new MecanumTuner();
}
```

Practical tips (not from the Pedro docs): several procedures drive the robot forward, left, or in place, and the robot coasts past its target. Clear a few feet around it, keep the OpMode stop button within reach, and use a fully charged battery. Tune on the same floor surface you will compete on.

Step 1: Drivetrain (Mecanum)

Set your drivetrain constants before tuning anything else. The Mecanum AutoTuner finds your `MecanumConfig`, which holds the four motor names and their directions.

1. Add the tuner to your `Tuning` class, then build and upload:

```
@Tuner
public static Procedure mecanumTuner() {
    return new MecanumTuner();
}
```

2. Open `http://192.168.43.1:10158` and select **Mecanum Procedure**.
3. Type the HardwareMap name of each drive motor in the field for its position. If your left front motor is configured as `left_front`, enter `left_front` in **Front Left HardwareMap Name**. Press **Submit** after all four.
4. Press **Continue**. A 3D diagram shows which way each wheel should spin, and the selected motor starts spinning.
5. Tell AutoTune whether the motor spun in the intended direction. Repeat for all four motors.
6. Open the **Java** tab and copy the generated `MecanumConfig` into `Constants.java`.

The result looks like this:

```

public static MecanumConfig drivetrainConfig = new MecanumConfig(
    c -> {
        c.frontLeftName.set("left_front");
        c.backLeftName.set("left_back");
        c.frontRightName.set("right_front");
        c.backRightName.set("right_back");

        c.frontLeftDirection.set(DcMotorSimple.Direction.REVERSE);
        c.backLeftDirection.set(DcMotorSimple.Direction.REVERSE);
        c.frontRightDirection.set(DcMotorSimple.Direction.FORWARD);
        c.backRightDirection.set(DcMotorSimple.Direction.FORWARD);

        // Optional: manual brake mode for TeleOp
        c.manualBrakeMode.set(true);
    }
);

```

Naming note: the Mecanum page of the docs calls this field `driveConfig`, while the Constants and Test pages call it `drivetrainConfig`. Pick one name and use it everywhere, including in `create()` and the `Tests` procedure.

Optional: test manual driving

Point the `Tests` procedure at your drivetrain only (the localizer and algorithm are `null`), build, open AutoTune, select **Tests** then **Driving**, and drive with a gamepad.

```

@Tuner
public static Procedure tests() {
    return new Tests(hardwareMap -> new Mecanum(hardwareMap,
        Constants.drivetrainConfig), null, null);
}

```

Step 2: Localizer (goBILDA Pinpoint)

The localizer tells the robot where it is on the field. If you have no odometry hardware you can fall back to drive motor encoders, but Pinpoint is far more accurate. The Pinpoint AutoTuner finds your `PinpointConfig`: pod directions, pod offsets, and pod resolution.

Check the hardware first. Two odometry pods must be connected, one parallel to the chassis length (forward) and one perpendicular to it (lateral). The three most common mistakes:

- The Pinpoint must be on an I2C port **other than port 0**, because the Control Hub's built-in IMU uses port 0.
- The Pinpoint must sit with the sticker side (the side with the ports) facing up.
- The forward pod plugs into the **x** port and the strafe pod into the **y** port.

Run the tuner

1. Add the tuner, then build and upload:

```
@Tuner
public static Procedure pinpointTuner() {
    return new PinpointTuner();
}
```

2. Open `http://192.168.43.1:10158` and select **Pinpoint Procedure**.
3. Enter the Pinpoint's HardwareMap name (for example `pinpoint`) and pick your pod type, then press **Submit**. The Four Bar and Swing Arm pods are goBILDA pods; for any other pods choose **Custom**.
4. **Custom pods only:** enter the distance you will push the robot (default 48 inches), start the test, push the robot forward that distance, then stop the OpMode. This measures the encoder resolution.
5. **Forward direction:** press **Continue**, push the robot straight forward, then stop the OpMode. AutoTune sets the x pod to `FORWARD` or `REVERSED`.
6. **Strafe direction:** press **Continue**, push the robot to the **left** (not the right), then stop the OpMode. AutoTune sets the y pod direction.
7. **Offsets:** press **Continue**, rotate the robot in place 180 degrees counterclockwise, then stop the OpMode. AutoTune calculates the x and y pod offsets.
8. Open the **Java** tab and copy the generated `PinpointConfig` into `Constants.java`. It contains the HardwareMap name, pod type, x and y pod directions, and x and y pod offsets.

The pod offsets are in inches and measure where each pod sits relative to the robot's center of rotation. If you would rather measure them by hand, the Pinpoint page of the docs has an offset diagram.

Localization test

Point `Tests` at your tuned drivetrain and localizer. The algorithm stays `null` for now.

```
@Tuner
public static Procedure tests() {
    return new Tests(
        hardwareMap -> new Mecanum(hardwareMap, Constants.drivetrainConfig),
        hardwareMap -> new PinpointLocalizer(hardwareMap,
Constants.localizerConfig),
        null);
}
```

Build, open AutoTune, select **Tests** then **Localization**, and drive or push the robot. The pose should start at (0, 0). Moving forward must increase `x`, and strafing left must increase `y`. If either axis moves the wrong way, redo the matching direction step.

Step 3: Foresight

Foresight is Pedro's built-in path-following algorithm. Its AutoTuner produces a `ForesightConfig` with your robot's maximum achievable velocities, natural decelerations, brake coefficients, and kP values. It needs your tuned localizer and drivetrain, so finish Steps 1 and 2 first.

1. Add the tuner. Confirm the localizer and drivetrain in it match your robot, then build and upload:

```
@Tuner
public static Procedure foresightTuner() {
    return new ForesightTuner(
        hardwareMap -> new PinpointLocalizer(hardwareMap,
Constants.localizerConfig),
        hardwareMap -> new Mecanum(hardwareMap,
Constants.drivetrainConfig));
}
```

2. Open `http://192.168.43.1:10158` and select **Foresight Procedure**.
3. Work through the sub-procedures below in order. Each one runs, then stops the OpMode.
4. Open the **Java** tab and copy the generated `ForesightConfig` into `Constants.java`.

#	Procedure	What the robot does	What it determines
1	Forward Velocity	Drives forward the distance you set, then stops; it will drift past the distance	Maximum forward velocity
2	Strafe Velocity	Drives left the distance you set, then stops; it will drift past	Maximum strafe velocity
3	Forward Deceleration	Drives forward until it reaches the velocity you set, usually a very short distance	Natural forward deceleration
4	Strafe Deceleration	Drives left until it reaches the velocity you set, usually a very short distance	Natural strafe deceleration
5	Heading Braking	Stays in place, spinning back and forth	Heading brake coefficients
6	Heading Tuner	Stays in place and spins one direction for a couple of seconds	Heading kP
7	Forward Braking	Drives forward a set distance and brakes, then returns to the start; repeats at several powers	Forward brake coefficients
8	Strafe Braking	Drives left a set distance and brakes, then returns to the start; repeats at several powers	Strafe brake coefficients
9	Forward Translational	Drives forward about 12 to 24 inches, then stops	Forward primary and secondary kP, plus coast and brake kV
10	Strafe Translational	Drives left about 12 to 24 inches, then stops	Strafe primary and secondary kP

Drifting past the target distance in the velocity procedures is expected, not a fault. Leave enough room for the robot to coast.

When the config is pasted in, move on to assembling `Constants.java` and testing.

Step 4: Assemble Constants.java

After tuning, `Constants.java` holds three groups of constants, plus a `create` method that builds your `Follower`.

- **Foresight (algorithm) constants:** the values from the Foresight AutoTuner, plus your path constraints.
- **Drivetrain constants:** motor names and directions for a mecanum drive.
- **Localizer constants:** the Pinpoint name, pod directions, and pod offsets.

The skeleton below shows the structure. The numbers in angle brackets are placeholders for what AutoTune generated for your robot; never reuse another robot's values.

```
package org.firstinspires.ftc.teamcode.pedro;

import com.pedropathing.algorithm.Foresight;
import com.pedropathing.algorithm.ForesightConfig;
import com.pedropathing.follower.Follower;
import com.pedropathing.revhub.drivetrains.Mecanum;
import com.pedropathing.revhub.drivetrains.MecanumConfig;
import com.pedropathing.revhub.localizers.PinpointConfig;
import com.pedropathing.revhub.localizers.PinpointLocalizer;
import com.qualcomm.hardware.gobilda.GoBildaPinpointDriver;
import com.qualcomm.robotcore.hardware.DcMotorSimple;
import com.qualcomm.robotcore.hardware.HardwareMap;

public class Constants {
    public static MecanumConfig drivetrainConfig = new MecanumConfig(
        c -> {
            c.frontLeftName.set("<name>");    // from Step 1
            c.backLeftName.set("<name>");
            c.frontRightName.set("<name>");
            c.backRightName.set("<name>");
            c.frontLeftDirection.set(DcMotorSimple.Direction.
<REVERSE|FORWARD>);
            // ...other three directions
            c.manualBrakeMode.set(true);      // optional, for TeleOp
        });

    public static PinpointConfig localizerConfig = new PinpointConfig(
```

```

        c -> {
            c.name.set("<pinpoint name>");    // from Step 2
            c.xPodOffset.set(<inches>);
            c.yPodOffset.set(<inches>);
            c.xPodDirection.set(GoBildaPinpointDriver.EncoderDirection.
<FORWARD|REVERSED>);
            c.yPodDirection.set(GoBildaPinpointDriver.EncoderDirection.
<FORWARD|REVERSED>);
        });

        // Paste the full ForesightConfig generated in Step 3 here.
        // It sets: forwardTranslational, strafeTranslational, coast, brake,
        // headingFeedback, headingBrakeCoefficients, linearBrakeCoefficients,
        // quadraticBrakeCoefficients, maxAchievableForwardVelocity,
        // maxAchievableStrafeVelocity, naturalForwardDeceleration,
        // naturalStrafeDeceleration.
        public static ForesightConfig foresightConfig = new ForesightConfig(c ->
{ /* generated */ });

        public static Follower create(HardwareMap h) {
            return new Follower(
                new PinpointLocalizer(h, localizerConfig),
                new Mecanum(h, drivetrainConfig),
                new Foresight(foresightConfig));
        }
    }
}

```

The localizer and drivetrain passed into `new Follower(...)` must match the hardware you actually tuned. If you ever switch localizer or drivetrain type, change both the config and the `create` method.

Step 5: Test the full Follower

The Tests procedure confirms the tuned Follower works. Update `tests()` to use all three parts (drivetrain, localizer, and algorithm), then build and upload.

```
@Tuner
public static Procedure tests() {
    return new Tests(
        hardwareMap -> new Mecanum(hardwareMap, Constants.drivetrainConfig),
        hardwareMap -> new PinpointLocalizer(hardwareMap,
        Constants.localizerConfig),
        () -> new Foresight(Constants.foresightConfig));
}
```

Open <http://192.168.43.1:10158>, select **Tests Procedure**, and run the tests below. A good order is Pose, Hold, Line, Curved, then Interpolation.

Test	What it does	Use it to check
Localization	Manual drive plus live position tracking	Axis directions and tracking drift
Driving	Manual drive only	Motor directions
Pose	Shows the localizer's pose in telemetry	The localizer alone, without driving
Hold	Holds the robot in place	Correction when you push it off course
Line	Drives forward and back along a straight line, 48 inches by default, until you stop it	Straight-line accuracy and stopping
Curved	Loops a quarter circle with tangential heading to (distance, distance), front and left of the start, then back; 48 inches by default	Curve following
Interpolation	Repeats the Curved path with various heading interpolations	Heading behavior on curves

When the Line test ends at the same spot every lap and the Hold test pulls the robot back after a push, the tuning is done. Next, set up your autonomous and create paths.

Checklist and common mistakes

- ☐ Pedro Pathing is installed and the project builds.

- ☐ Mecanum tuner done; the four motors spin in the intended directions; `drivetrainConfig` pasted into `Constants.java`.
- ☐ Pinpoint is on an I2C port other than 0, sticker side up; forward pod on x, strafe pod on y.
- ☐ Pinpoint tuner done, with the robot pushed **left** for the strafe step; `localizerConfig` pasted in.
- ☐ Localization test: pose starts at (0, 0); forward raises `x`; left raises `y`.
- ☐ All ten Foresight sub-procedures run; `foresightConfig` pasted in.
- ☐ `create()` uses the same localizer and drivetrain you tuned.
- ☐ `tests()` uses all three parts; Line, Curved, and Hold tests behave.

If something looks wrong, check the Pedro Pathing troubleshooting page first.

Sources

All steps are based on the official Pedro Pathing docs (Pedro 3), opened on 2026-10-09:

- [Tuning overview](#)
- [Mecanum drivetrain](#)
- [Localization](#)
- [Pinpoint localizer](#)
- [Foresight](#)
- [Constants](#)
- [Test](#)